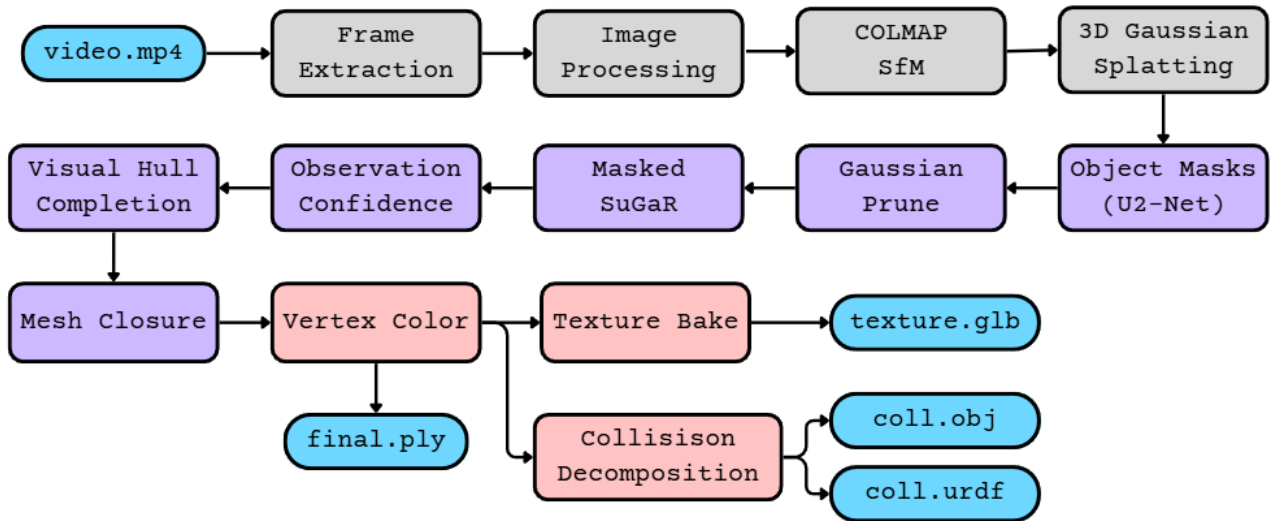


SuGaRrush: Fast SuGaR for object reconstruction

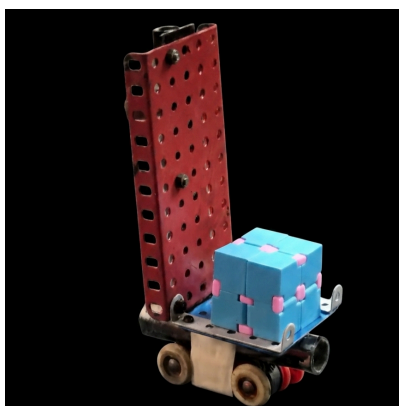
Syed Adeel Ahmad

Architecture:

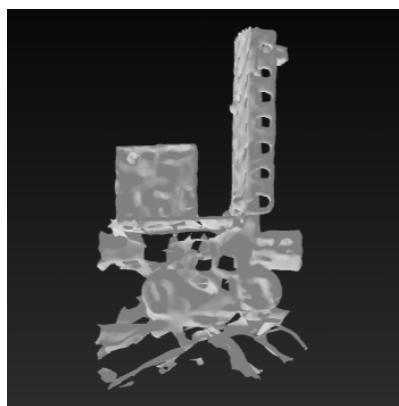


I built the pipeline in multiple sequential iterations and will explain them in order:

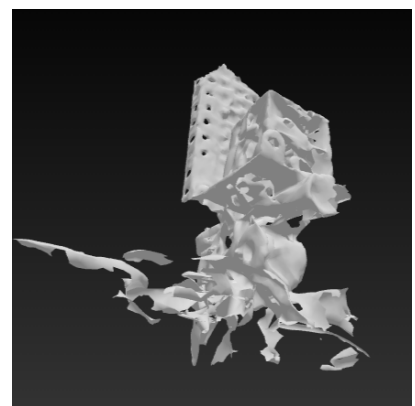
- As outlined in the task doc, i started off with basic SfM + SuGaR mesh extraction. I set up the entire thing on my local system with a GTX 1650 (4GB). Raw frames are processed from the captured video and fed into COLMAP which outputs camera poses and sparse point cloud. This output was then used to train and render 3D Gaussians. Then the gaussians are aligned to the surface as per sugar and a mesh is extracted and using poisson reconstruction followed by long refinement. After 12 hours, the results were fine for a first iteration but obviously this wasnt a watertight textured mesh that you are expecting with an entire mesh over the scene which is a waste since the task was "object" reconstruction. I used Open3D to separate the object by cutting it along planes, which resulted in large gaps, especially in the unobserved underside of the mesh.



a) Object 1 Input frame



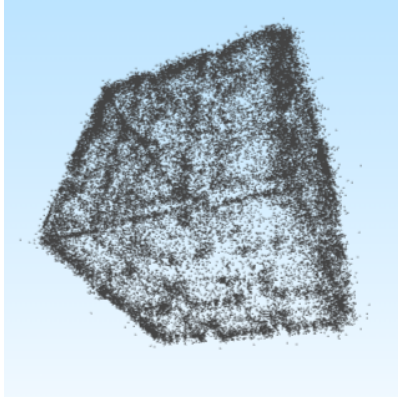
b) Object 1 mesh (side)



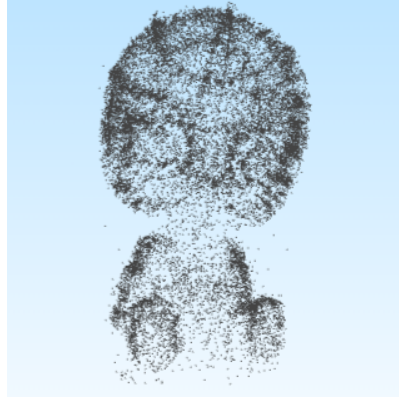
c) Object 1 mesh (underside)

- Next, I automated the last part, that is isolating the object from the scene and used a deep learning model U2-Net (its not inventing geometry, just labelling) which outputs a mask around the detected object, but it also resulted in the mesh being open from multiple viewpoints. Also it made no sense to spend the compute budget on meshing the entire scene. So the solution was to apply the masking before the mesh was built. I first applied the masks just after frames were processed and then only fed the masked images into COLMAP. But i quickly realized my stupidity as masking only objects strips the SfM from matching features provided by the background and surrounding, which

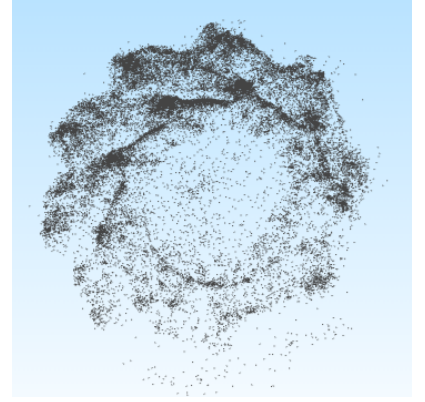
resulted in poor camera pose localization. Similar logic applies to 3DGS training as well. So, the right way to apply masking is after the gaussians have been trained and before they are fed into SuGaR for surface alignment and meshing. Here are some isolated gaussians:



a) Isolated Rubik's cube

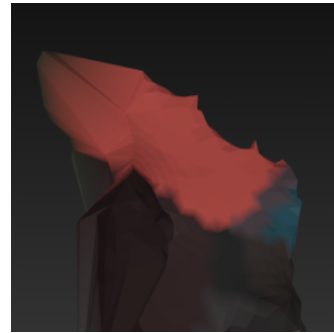
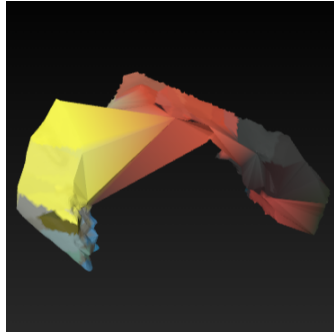


b) Isolated spiderman figure



c) Isolated Mecanum wheel

- This seemed way easier than it actually was. I started by applying the mask for every camera pose and pruned all gaussians laying outside of the mask and handed them over to SuGaR, which obviously, failed catastrophically and sent me into hours of bug hunting. I found that the photometric loss in SuGaR reoptimizes its input Gaussians against the ground truth images, which consist of the full scene as handed over to COLMAP and also treats the gaussians as learnable, the remaining object gaussians were free to move and enlarge in an attempt to explain all those missing background pixels. And they became elongated structures, spikes, sheets and hooks around the object.



- The fix was to reconstruct the photometric loss to let the gaussians explain only the foreground object, remain near the pruned object, and receive high penalty for leaving the mask. The pruned gaussians here did not have alpha/opacity values, so i used the color values to create a sort of visual hull over the mask. There's this classic technique from graphics called black composite, which allows me to process the background as is, but penalises any gaussian from leaving the mask boundaries. It transforms each ground truth image by multiplying it by the binary object mask:

$$I_{target}(u, v) = M(u, v) \cdot I_{RGB}(u, v) \quad \text{where } M(u, v) \in \{0, 1\}$$

The object retains its original RGB values and every background pixel becomes black. The full loss:

$$L1(R_{\theta}, I_M) = \sum_{v=1}^H \sum_{u=1}^W \sum_{c=1}^3 |R_{\theta, c}(u, v) - M(u, v) \cdot I_c(u, v)|$$

$$\Rightarrow L1(R_{\theta}, I_M) = \sum_{v=1}^H \sum_{u=1}^W \sum_{c=1}^3 \{ \sum_{M=1} |R_{\theta, c}(u, v) - I_c(u, v)| + \sum_{M=0} |R_{\theta, c}(u, v)| \}$$

Inside the mask, $M=1$, so loss is $|R-I|$ and outside its $|R|$, thus object silhouette is penalized.

Decision: A strictly masked L1 would be: $L_{masked} = \sum M |R_{\theta} - I| / 3 \sum M$, so outside the mask:

$M |R_{\theta} - I| = 0$. Therefore, it supplies neither reward nor penalty for geometry rendered outside the silhouette, while black composited mask gives $|R_{\theta}|$ which actively suppresses opacity and colour.

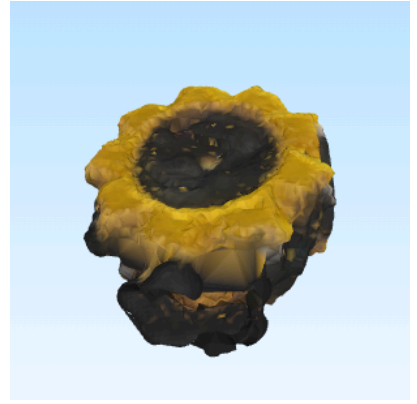
We can see the initial coarse mesh here (notice the black color blobs in the underside):



a) Rubik's cube

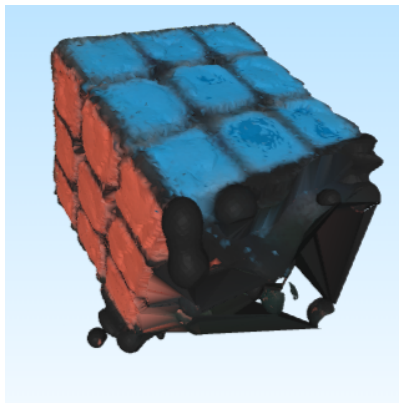


b) spiderman figure



c) Mecanum wheel

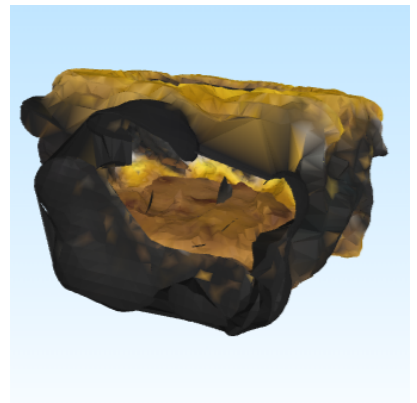
- Now, the gaussian prune isolates the object in gaussian space, but it never checks the resulting mesh against the cameras, and so geometry in a region no camera ever saw survives all the way to the end: detached blobs floating beside the object and spikes coming off the unobserved underside. Poisson reconstruction cannot tell that junk from a face that was seen multiple times, it fits both with the same weight. Here are few examples of the underside of coarse mesh generated after Sugar:



a) Rubik's cube



b) spiderman figure



c) Mecanum wheel

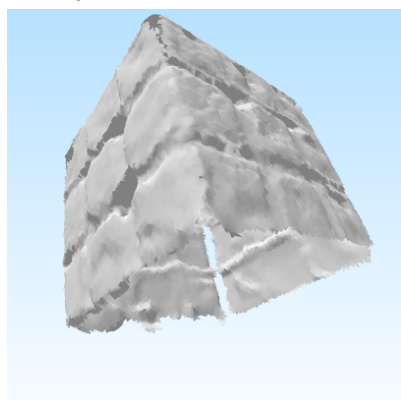
- Observation confidence filtering: I spent a while tuning octree depth and cleanup radii before accepting that this might not be a meshing parameter. A face that no camera ever looked at is not evidence about the object, and maybe the right thing to do is measure that directly and delete it. So i compute a per face observation confidence p from the actual cameras and the same U2-Net masks. For a face f with centroid c and normal n , over the camera set C :

$$\rho_{acc}(f) = \sum_c vis(f, c) \cdot M_c(\pi_c(c)) \cdot |\langle n, d_c \rangle|$$

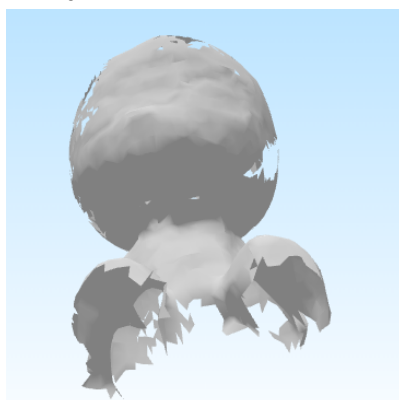
$$\rho(f) = \min(\rho_{acc}(f) / \kappa, 1)$$

where, $vis(f, c)$ is the depth buffer visibility of the face in camera c , M_c is the mask value at the projected centroid, d_c is the unit direction from the centroid to the camera centre, and κ is a normaliser so that a comfortably covered face reaches 1. I also keep two counters per face, n_{vis} (visible **and** inside the silhouette) and n_{geom} (visible at all, mask ignored). κ is not a constant, a hardcoded one would not transfer across view counts or object scales, so it is taken as the 75th percentile of ρ_{acc} over the mesh. A face is seeded as junk if it belongs to a connected component smaller than 2% of the mesh that fewer than 20% of the evaluated views see, or if it is geometrically visible in at least 2 views but agrees with the silhouette in fewer than a keep-ratio fraction of them ($n_{vis} / n_{geom} < 0.75$ by default).

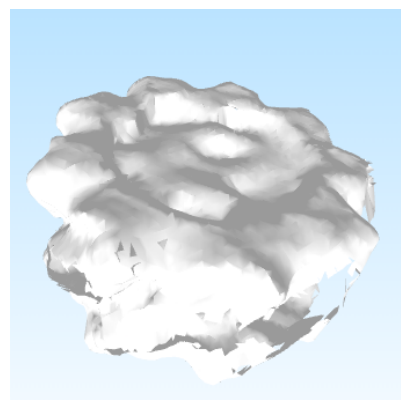
Here you will see that the unobserved junk is cleaned up:



a) Rubik's cube

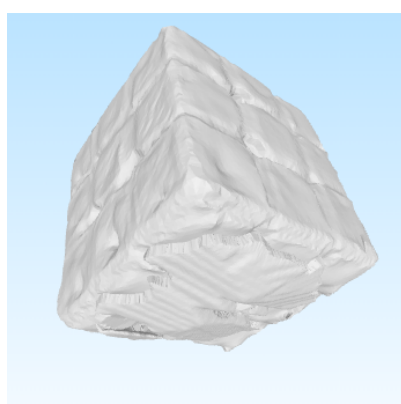


b) spiderman figure

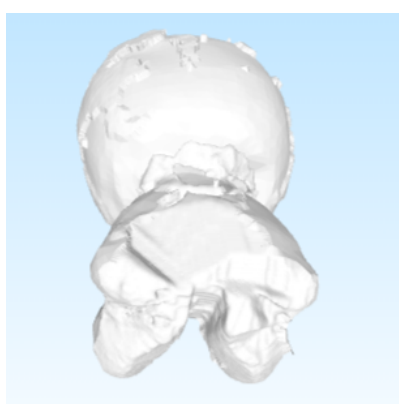


c) Mecanum wheel

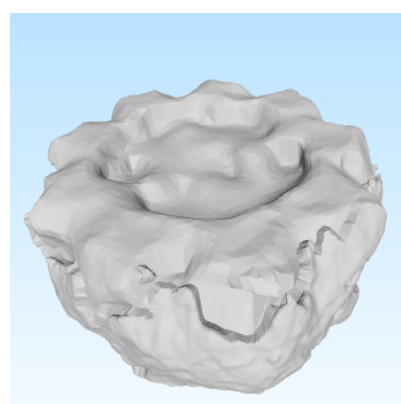
- Mesh closure (finally!): ρ leaves the mesh more correctly observed but quite open. I tried multiple strategies over the Rubik's cube to fill the open unobserved side. A Poisson refit minimises a smooth energy, so its minimiser over a large unobserved face is a dome. A least squares planar cap fits one plane to the rim and returns a plate but sliced a corner off the cube. To be honest, I felt out of expertise in this stage and so, I relied on someone else fixing my issues- Marco Attene's glorious pymeshfix. I wasn't getting the expected results from it, as it was shrinking the total object volume and axes. I diagnosed that pymeshfix repairs by deleting, so its damage scales with the topological junk it is handed over. So the answer was to hand pymeshfix a clean shell with one small hole in it, instead of a lacy one with a huge hole which meant filling the big unobserved face myself first, with something that isn't a guess. The visual hull is the only option that qualifies, because it is built purely from the silhouettes the cameras already gave me and so cannot add anything they didn't supply (it invents nothing, same as U2Net just labelling). The construction is simple: grow the box around the object by 15%, cut it into a 256^3 grid of little cubes, ask ~ 120 views whether each cube falls inside the object outline, and keep what survives. I take the outer skin of that surviving blob as points, throw away the parts sitting over surface I had already measured, keep only what lands in the hole, mix it with 200k points sampled off the measured surface, and rebuild one continuous surface over the union and then snap the measured vertices back to exactly where they were. Here's the final fixed mesh from underside:



a) Rubik's cube



b) spiderman figure



c) Mecanum wheel

- Texture baking: SuGaR does export a textured OBJ, but that texture is made by rendering the refined SuGaR appearance and averaging it into per triangle UV squares, so it is a picture of the model, not of the object, and it also does not exist on the final closed mesh which has been through carving, completion and repair. So the atlas is baked independently, straight from the original undistorted frames, and colour comes only from camera. The mesh is unwrapped with xatlas onto a flat 2048^2 sheet, the atlas is rasterized in UV space with nvdiffrast, and then every

texel is projected into every source view using the exact COLMAP intrinsics. It works as a vote again. a frame only gets a vote on that pixel if it could really see it, not off the edge of the image, not outside the object mask (masks get shrunk 2 px first, otherwise background colour bleeds onto the silhouette), not viewing the surface nearly edge on (a grazing view carries almost no colour), and not blocked by another part of the object, which i check against a depth render. The best 24 surviving votes are kept, ranked by how square-on the view is, and then combined with a median instead of an average.

- There is a collision decomposition step in the pipeline as well that runs CoACD convex decomposition over the watertight mesh and emits a compound convex collision asset plus a URDF, ready to drop into Gazebo or Issac. This stage invents nothing either, it strictly bounds the shell that already exists in convex hulls. I did not manually implement the code either.

Speed & Memory Optimizations:

- Error guided SDF Sampling: Initially the most time consuming part of the pipeline is the SuGaR mesh extraction phase. During the SDF regularization step, sugar treats the gaussian cloud as a continuous density field and it generates random 3D points to regularize that field. The default sample size for each iteration was $N_{\text{samples}} = 1\text{M}$. The SDF loss is computed as a mean over sampled points, not as sum. Changing N therefore does not directly change the expected magnitude of the loss. It mainly changes the variance of its Monte Carlo estimate. And because the optimizer repeats this process over thousands of steps, some of the per step noise averages out across training. A lower uniform sample count may undersample geometrically difficult regions such as thin structures, sharp corners, high curvature areas, etc. Uniform sampling would continue allocating most of the budget to these easy regions because they make up most of the representation and difficult regions may remain inaccurate. I solved this allocation problem using error guided sampling, found on a relevant old stack exchange article. In error guided SDF sampling, difficult Gaussians receive more samples, while a uniform component still preserves global coverage. For Gaussian g , the implementation maintains an exponential moving average as residual score:

$$Eg(t) = \beta Eg(t - 1) + (1 - \beta)Rg(t)$$

where, $Eg(t)$ is the stored error score, $Rg(t)$ is the newly observed residual, β is the EMA decay. The per gaussian EMA values are normalized into an error based probability distribution as,

$$p(g) = (1 - m)p_{\text{base}}(g) + mp_{\text{error}}(g)$$

$p_{\text{base}}(g)$ be the ordinary sampling probability, $p_{\text{error}}(g)$ be the normalized residual EMA probability. m is the configured error mixture. Using this technique, i was able to reduce $N = 50\text{k}$ safely.

- Sugar's camera wrapper moves every ground truth image onto CUDA at construction time and keeps it resident for the entire run, so its persistent footprint is

$$\text{bytes} = n_{\text{images}} \cdot w \cdot h \cdot 3 \text{ channels} \cdot 4 \text{ bytes}$$

which is linear in frame count. I fixed the resolution cap & tuned for 240 frames which went OOM at 500 (which took prayers to revive my laptop). So the cap is derived instead of chosen: measure free VRAM at launch, subtract SuGaR's non-GT footprint (model + optimizer + rendering, measured at ~2.5 GB peak here, reserved as 2.8), clamp the remainder into a [0.6, 1.4] GB image budget, and solve the footprint equation to get image side:

$$\text{side} = \sqrt{(\text{budget_bytes} / (12 \cdot n_{\text{images}} \cdot h/w))}$$

That gives 768 px for 236 frames (0.94 GB resident), 576 px for 400 frames (0.89 GB) and 544 px for 499 frames (~0.9 GB). After this my pipeline has never reached OOM.

- Focal Cropping: Because we have already pruned most of the gaussians, we do not need to see most of the background during the Sugar's mesh extraction phase since the object is only ~5% of a frame at that cap. So, for each camera, take the foreground bounding box from the mask, centre a fixed $S \times S$ window on it at full source resolution ($S = 384$ by default, or the smaller original image

dimension if that is less), and clamp the window to the image. The key point that makes this legal is that a crop shifts only the principal point, it does not change the focal length:

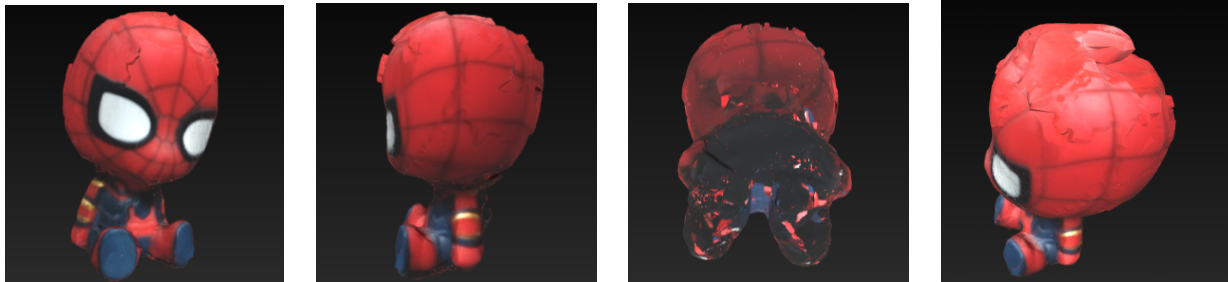
$$c_x = w_{original}/2 - x_0, \quad c_y = h_{original}/2 - y_0, \quad FoV = 2 \cdot \arctan(S / 2f)$$

so every camera keeps a uniform image size and focal and only c_x , c_y differ per view, which the rasterizer already represents through its projection matrix, and the shift propagates cleanly through the PyTorch3D camera conversion. If a view has no foreground at all, it crops around the image centre and carries on. This results in about 4× the effective object resolution at the same memory cost.

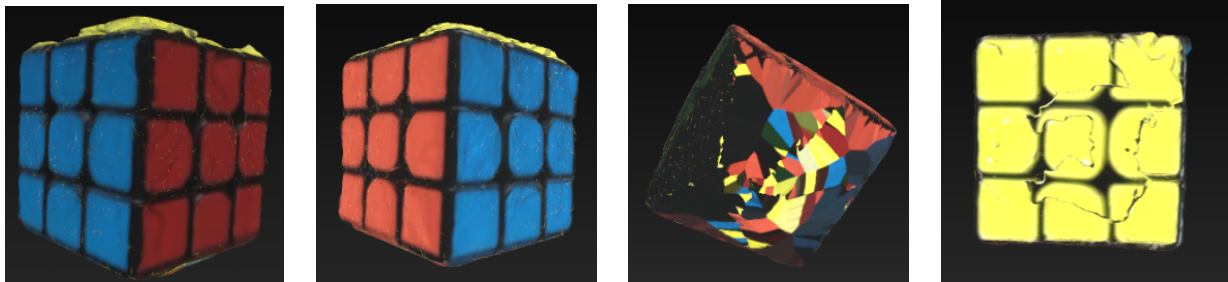
By using these approaches I was able to successfully train and test multiple iterations of each object with different parameters without OOM or time budget issues.

And now we have the final renders of the textured watertight object meshes -

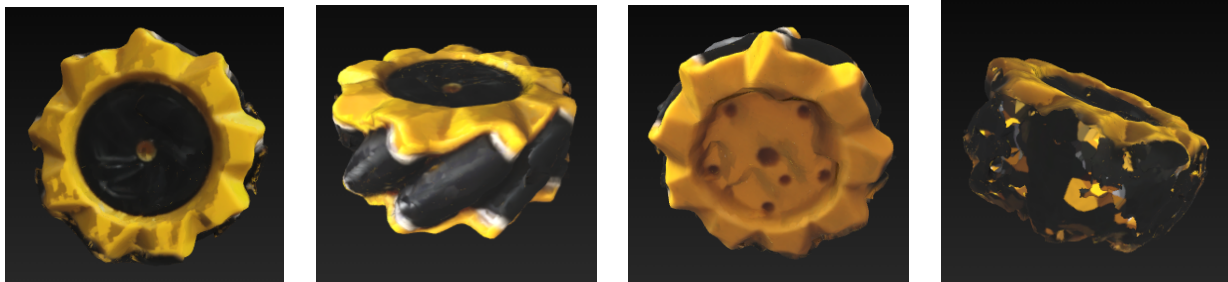
1) Spiderman Wobblehead figure:



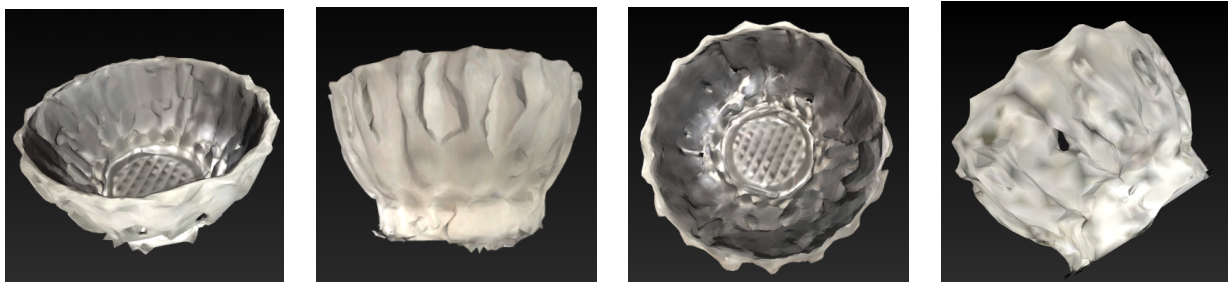
2) Rubiks Cube (3x3):



3) Mecanum Wheel:



4) Glass Bowl:



Assessment of output meshes:

- Spiderman figure: The mesh reconstructs fully watertight, but the textures are not clearly visible on the head. The reflected lighting from my ceiling have been baked permanently into the texture. Also, on the top of the head, some area is bludgeoned inside. I checked the footage and this is the area that goes out of frame for a considerable part. So the observation confidence step takes out some faces from this part and then pymeshfix fills in a smooth curve instead of the crown. Also, the costume's web lines and the hands are gone, at the derived resolution and the object's size in frame there wasn't enough signal for detail at that scale.
- Rubik's Cube: Geometrically this is the strongest output the pipeline produced, almost a perfect cube. The texture is also well with the correct sticker colors and black separation lines. But the edges and corners are visibly rounded and the faces are slightly pillowed rather than flat. Also there is an island of lump over the yellow face. The observation confidence could not remove it because the gaussians might have looked similar against the ground truth photos. This also means that the pipeline has literally no idea about the actual structure of what it's observing. These kinds of issues could be dealt by finding an axis of symmetry between the individual faces of the cube and observing only the mean structure between them.
- Mecanum Wheel: A Mecanum wheel is a rim of angled barrel rollers with air between them and a hub with a through-hole. The outer diameter and the overall disc silhouette are right, the yellow and black material split is in the correct places, and it is a valid watertight solid. But there are a lot of major issues. It has through hole in the center for axle joint, but it is clearly missing from the mesh. It might be due to no or small number of shots with the through hole geometry visible. Also the gap between two rollers is not visible as background in any silhouette from an orbit, because from every direction another roller is behind it, so the hull votes those voxels occupied, the screened Poisson fuses across them, and the rollers come out as one continuous lump of barrels.
- Glass bowl: 3DGS reconstructs a view dependent radiance field and for some transparent surface the radiance at a point is mostly whatever is behind it which changes with viewpoint, and so there is no consistent surface for the density field to place a level set on. So the mesh comes out to be lumpy and almost textureless. Tho the exterior comes out somewhat better, the vertical ridges are resolved and the wall thickness and rim diameter look plausible. There is also a see through hole in the final bowl, most probably because the physical region let light through exactly in majority of the camera poses in which it is being observed.

What I would do with one more week:

I would immediately focus on improving the mesh quality and getting my own solution for the unobserved regions. Firstly, I would like to experiment with 2d sugar, i believe it would be better at surface representation. Secondly, as observed from the results i got, i would think of some way to extract connectivity and symmetric relationships for each object because we now have a 3d representation of the object instead of a 2d image. I got this idea from Y.wan's paper "Symmetry Hierarchy of Man-Made Objects" and a couple more papers for understanding structure from 3d representation without using generative feed forward network, though most methods use diffusion models, i believe for the strict non-generative constraint, a symmetry hierarchy would provide additional information about the structure. I can discuss the exact details in person better.

END OF REPORT